Звіт про виконання завдання № 5.3

Функції, що містять розгалуження та цикли з рекурентними співвідношеннями

Варіант 7

Виконав: Джамалов Олександр Олегович, група PI-11

Мета роботи: навчитися використовувати функції.

Умова завдання (фрагмент сторінки 67 посібника)

Варіант 7.

Обчислити і вивести на екран у вигляді таблиці значення виразу від дійсного z на інтервалі від $z_{поч}$ до $z_{кін}$ з n відрізками розбиття інтервалу. Параметри zпоч, zкін, n вводяться з клавіатури. Таблиця має містити заголовок та шапку. Кожний рядок таблиці має містити значення z та значення виразу. Використовувати допоміжний алгоритм, реалізований за допомогою окремої функції. При обчисленні значення доданків в сумі використовувати рекурентні співвідношення.

$$k(z^2+1) - k(z^2-1) + 2k(z) \quad , \quad де$$

$$k(x) = \begin{cases} \dfrac{e^x + \sin x}{\cos^2 x + 1} & , \quad |x| \geq 1 \\ \dfrac{1}{e^x} \displaystyle\sum_{i=0}^{5} \dfrac{x^i}{i!} & , \quad |x| < 1 \end{cases}$$

Рекурентне співвідношення

a_0 = 1; a_i = a_(i-1) · x / i, i = 1, 2, 3, 4, 5.

Оскільки a_i = x^i / i!, відношення сусідніх доданків дорівнює x / i.

Отже, R_i = x / i; сума містить 6 доданків, включно з a_0.

Для |x| < 1 функція повертає sum / exp(x).

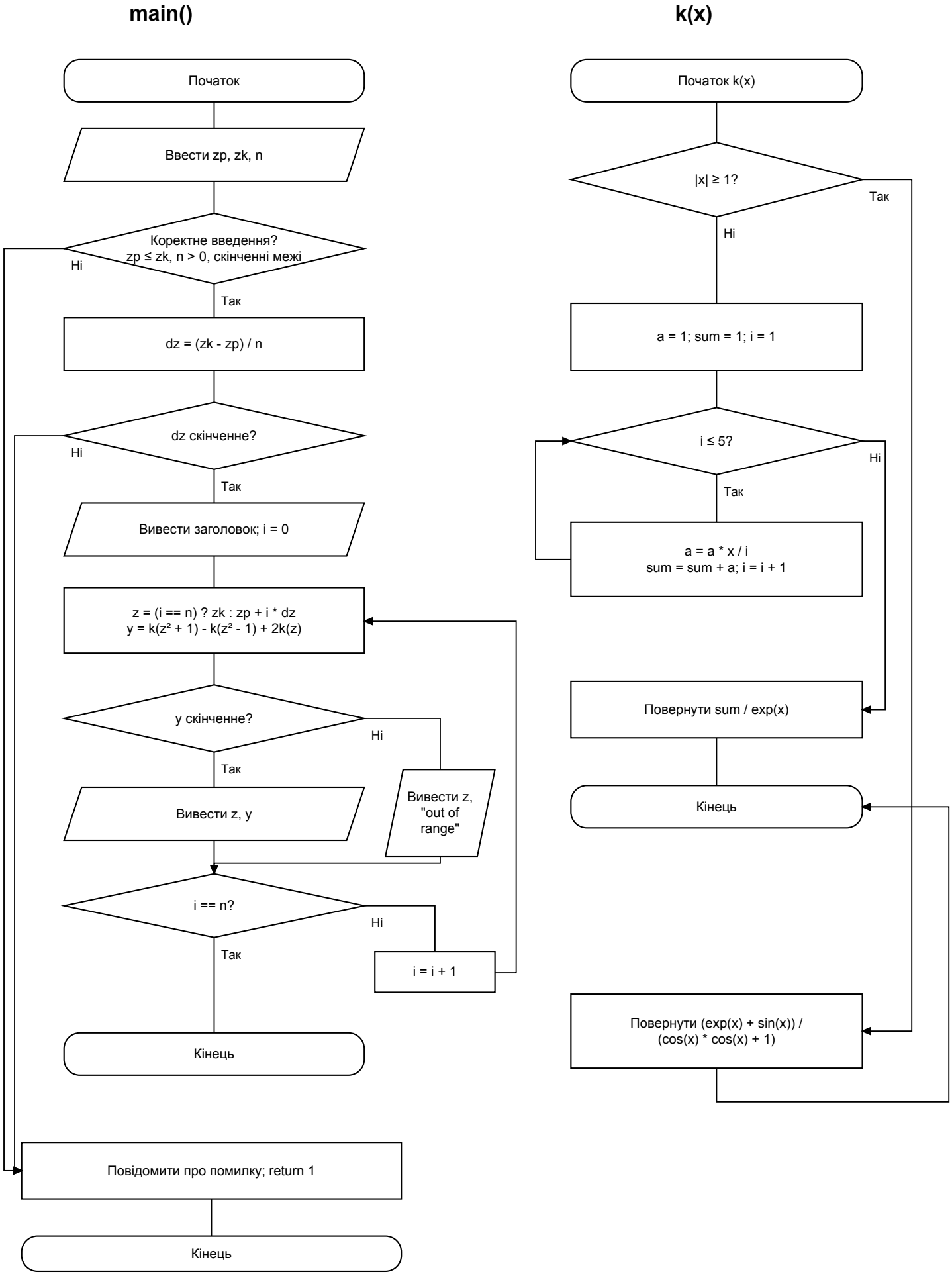
Крок таблиці: Δz = (zкін - zпоч) / n. Кількість рядків: n + 1.

Тригонометричні функції використовують радіани.

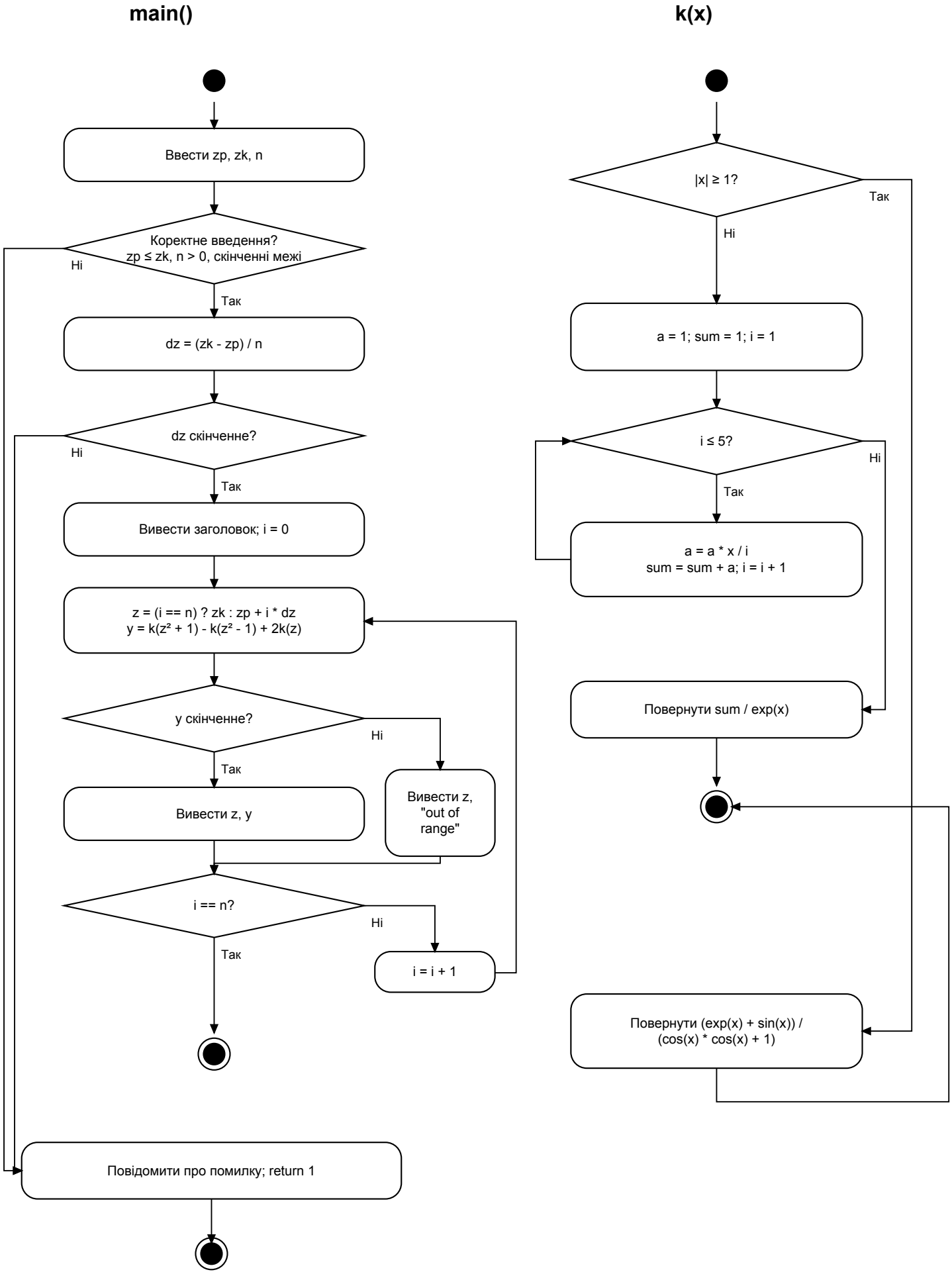
Вхідні дані: zпоч ≤ zкін, ціле n > 0; межі мають бути скінченними.

Джерело: Григорович В. Г., інструкції до лабораторної роботи № 5, 2023.

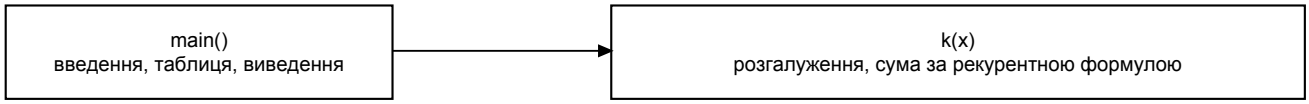
Блок-схеми алгоритмів



UML-діаграми діяльності



Структура програми та основна програма



main() викликає k(x) тричі для кожного значення z.

Файл lab_5_3.cpp (основна програма)

```
#include <cmath>
#include <iomanip>
#include <iostream>

using namespace std;

double k(double x);

#ifndef UNIT_TEST
int main()
{
    double zp, zk;
    int n;
    cout << "z_begin z_end n: ";
    if (!(cin >> zp >> zk >> n) || !isfinite(zp) ||
        !isfinite(zk) || zp > zk || n <= 0)
    {
        cerr << "Invalid input: z_begin <= z_end, n > 0.\n";
        return 1;
    }

    const double dz = (zk - zp) / n;
    if (!isfinite(dz))
    {
        cerr << "Interval is too large.\n";
        return 1;
    }

    cout << "Lab 5.3 / Variant 7\n"
         << "F(z) = k(z*z+1) - k(z*z-1) + 2*k(z)\n"
         << setw(16) << "z" << setw(22) << "F(z)" << '\n';
    cout << fixed << setprecision(8);

    // n intervals have n + 1 nodes, including both endpoints.
    for (int i = 0; ; ++i)
    {
        const double z = (i == n) ? zk : zp + i * dz;
        const double y = k(z * z + 1) - k(z * z - 1) + 2 * k(z);
        cout << setw(16) << z;
        if (isfinite(y))
            cout << setw(22) << y << '\n';
        else
            cout << setw(22) << "out of range" << '\n';
        if (i == n)
            break;
    }
    return 0;
}
#endif
```

Допоміжна функція та unit-тести

Файл lab_5_3.cpp (функція k)

```
// Auxiliary function from the assignment.
double k(double x)
{
    if (abs(x) >= 1)
        return (exp(x) + sin(x)) / (cos(x) * cos(x) + 1);

    double a = 1; // a_0 = x^0 / 0! = 1
    double sum = a;
    for (int i = 1; i <= 5; ++i)
    {
        a *= x / i; // a_i = a_(i-1) * x / i
        sum += a;
    }
    return sum / exp(x);
}
```

Файл test_5_3.cpp

```
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>

#define UNIT_TEST
#include "lab_5_3.cpp"

int main()
{
    // Independent reference: explicit polynomial, no recurrence.
    const double xs[] = {-2, -1, -0.999999, -0.5, 0,
                         0.5, 0.999999, 1, 2};
    int failed = 0;
    for (double x : xs)
    {
        double expected;
        if (std::abs(x) >= 1)
            expected = (std::exp(x) + std::sin(x)) /
                       (1 + std::pow(std::cos(x), 2));
        else
            expected = (1 + x + std::pow(x, 2) / 2 +
                        std::pow(x, 3) / 6 + std::pow(x, 4) / 24 +
                        std::pow(x, 5) / 120) / std::exp(x);

        const double actual = k(x);
        const bool ok = std::isfinite(actual) &&
            std::abs(actual - expected) <=
            1e-12 * std::max(1.0, std::abs(expected));
        std::cout << (ok ? "PASS" : "FAIL")
                  << " x=" << std::setw(10) << x
                  << " k(x)=" << std::setprecision(12)
                  << actual << '\n';
        if (!ok)
            ++failed;
    }
    std::cout << "Tests: 9, passed: " << 9 - failed
              << ", failed: " << failed << '\n';
    return failed == 0 ? 0 : 1;
}
```

Результати виконання та висновки

Перевірка функції k(x)

Компілятор: clang++, C++17; ключі -Wall -Wextra -Wpedantic.
Нижче наведено фактичний вивід виконаного тестового файла.

```
PASS x=        -2 k(x)=-0.659714057482
PASS x=        -1 k(x)=-0.366577753206
PASS x= -0.999999 k(x)=0.996703359754
PASS x=      -0.5 k(x)=0.99996662486
PASS x=         0 k(x)=1
PASS x=       0.5 k(x)=0.999985835063
PASS x=  0.999999 k(x)=0.999405818248
PASS x=         1 k(x)=2.7553832111
PASS x=         2 k(x)=7.0733956696
Tests: 9, passed: 9, failed: 0
```

Приклад: zпоч = -1, zкін = 1, n = 4

```
           z                F(z)
-1.00000000          5.34024016
-0.50000000          5.03825666
 0.00000000          5.12196096
 0.50000000          5.03829508
 1.00000000         11.58416209
```

Додатково пройдено 8 інтеграційних перевірок: 3 таблиці,
4 некоректні введення та 1 випадок переповнення.
Тести охоплюють x = -1, 0, 1, обидві гілки та точки біля меж.
Для суми еталон обчислено прямим поліномом, без рекурентності.
Допуск: 10^(-12) · max(1, |еталон|). Усі 9 unit-тестів пройдено.

Посилання на Git-репозиторій

Не надано. Додати URL після завантаження файлів проєкту.

Висновки

Реалізовано допоміжну функцію з розгалуженням за |x| та цикл обчислення суми. Рекурентна формула дозволяє отримувати доданки без окремого обчислення степенів і факторіалів. Основна програма виводить n + 1 значень виразу, включно з обома кінцями інтервалу. Коректність перевірено автоматизованими тестами. При переповненні типу double програма виводить повідомлення "out of range".